Exercícios de Arrays — Supermercado Bom Dia

Oct 19, 2026 · @Durvaldo

São 21 exercícios sobre arrays, todos passados no dia a dia do Supermercado Bom Dia e da sua equipe. Cada exercício traz o resultado esperado para a turma conferir com `console.log()`.

A equipe do Bom Dia

O Bom Dia tem 46 funcionários — a turma inteira. Vários exercícios usam o array abaixo; copie para o seu arquivo antes de começar.

```
const equipe = [
  "André Luiz", "Arthur", "Breno", "Carlos Eduardo",
  "Cauã", "Davi", "Davi", "Eduardo",
  "Emerson", "Erick", "Eris", "Ezequiel",
  "Felipe", "Francisco", "Gabrielly", "Guilherme",
  "Gustavo Alexandre", "Gustavo Augusto", "Heloa", "Isabelle",
  "Ivaneide", "Jayme", "Jorge", "João Pedro",
  "João Vitor", "Kaio", "Luis Otávio", "Layson",
  "Lorenzzo", "Lucas", "Lucas Matheus", "Luis Gustavo",
  "Luiz", "Magsdene", "Marco Antônio", "Mateus",
  "Nickson", "Octavio", "Otávio", "Patrick",
  "Pedro Junio", "Rafael", "Victor Hugo", "Victor Hugo",
  "Vinícius", "Willky",
];
```

Todos os exercícios usam apenas arrays simples, de textos, números ou `true` / `false` — nenhum objeto.

Nível 1 — Fundamentos

1. Escala da manhã

A escala do turno da manhã está assim:

```
const escalaManha = ["Heloa", "Erick", "Vinícius", "Davi", "Luiz"];
```

Exiba quem abre a loja (o primeiro), quem fecha o turno (o último) e quantas pessoas estão escaladas.

Resultado esperado: `"Heloa"`, `"Luiz"` e `5`.

2. Reforço no estoque

Chegou um caminhão de mercadorias e a equipe do estoque precisa de ajuda.

```
const equipeEstoque = ["Ezequiel", "Pedro Junio"];
```

Use `push()` para adicionar `"Nickson"`. Logo depois, o Nickson foi chamado para o caixa: use `pop()` para tirá-lo da equipe e exiba quem saiu.

Resultado esperado: `"Nickson"` saiu e a equipe volta a ser `["Ezequiel", "Pedro Junio"]`.

3. Fila do cafezinho

```
const filaCafe = ["Lucas", "Mateus", "Otávio"];
```

A gerente Isabelle chegou com pressa para uma reunião e foi colocada no início da fila com `unshift()`. Em seguida, a primeira pessoa da fila pegou seu café: remova-a com `shift()`.

Resultado esperado: `"Isabelle"` foi atendida e a fila fica `["Lucas", "Mateus", "Otávio"]`.

4. Churrasco de fim de ano

```
const confirmados = [
  "Gustavo Alexandre", "João Vitor", "Heloa", "Victor Hugo",
  "Breno", "Octavio", "Kaio", "Lorenzzo",
];
```

Use `includes()` para descobrir se a Heloa e o Pedro Junio confirmaram presença. Quem não estiver na lista deve ser adicionado ao final.

Resultado esperado: Heloa → `true`; Pedro Junio → `false`, e depois ele aparece no fim da lista.

Nível 2 — `map()`

5. Crachás dos novatos

O RH digitou os nomes dos novatos tudo em minúsculas:

```
const novatos = ["patrick", "magsdene", "willky"];
```

Use `map()` para gerar o texto de cada crachá, com o nome em maiúsculas.

Resultado esperado: `["CRACHÁ • PATRICK", "CRACHÁ • MAGSDENE", "CRACHÁ • WILLKY"]`

6. Reajuste do vale-alimentação

O vale-alimentação de Arthur, Cauã, Felipe e Ivaneide teve aumento de R$ 50.

```
const valesAtuais = [400, 450, 500, 380];
```

Crie um novo array com os valores reajustados, sem alterar `valesAtuais`.

Resultado esperado: `[450, 500, 550, 430]`, e `valesAtuais` continua igual.

7. Ordem da reunião

Na reunião de segunda, cada um apresenta os resultados da semana nesta ordem:

```
const apresentacoes = ["Isabelle", "Francisco", "Guilherme"];
```

Use o índice do `map()` para numerar as apresentações.

Resultado esperado: `["1º - Isabelle", "2º - Francisco", "3º - Guilherme"]`

8. Bom dia no grupo

Todo dia a gerência manda um bom-dia para cada funcionário no grupo da loja. Use `map()` em `equipe` para criar as mensagens.

Resultado esperado: um array de 46 textos começando com `"Bom dia, André Luiz!"`, `"Bom dia, Arthur!"`, `"Bom dia, Breno!"`...

Nível 3 — `filter()`

9. Dias de muito bolo

O Lucas anotou quantos bolos a padaria vendeu em cada dia da semana:

```
const bolosPorDia = [32, 18, 25, 40, 12, 27];
```

Filtre os dias em que a padaria vendeu pelo menos 25 bolos.

Resultado esperado: `[32, 25, 40, 27]`

10. Camisetas do time de futebol

A loja vai montar um time e as camisetas com a letra G chegaram primeiro. Use `filter()` em `equipe` para selecionar os nomes que começam com `"G"`. Dica: `startsWith()`.

Resultado esperado: `["Gabrielly", "Guilherme", "Gustavo Alexandre", "Gustavo Augusto"]`

11. Horas extras para assinar

Estas são as horas extras registradas no mês:

```
const horasExtras = [6, 2, 8, 0, 5, 7, 3, 9, 4, 10, 1, 6, 0, 12, 5, 2];
```

Registros entre 5 e 10 horas (inclusive) precisam da assinatura da gerente Isabelle. Filtre esses registros usando duas condições no mesmo `filter()`.

Resultado esperado: `[6, 8, 5, 7, 9, 10, 6, 5]`

> O 12 fica de fora porque passa de 10 — esse vai direto para o RH.

12. Nomes compostos no ponto

O relógio de ponto só mostra o primeiro nome, então quem tem nome composto precisa ajustar o cadastro. Use `filter()` em `equipe` para achar os nomes que têm espaço (dica: `includes(" ")`) e exiba quantos são com `length`.

Resultado esperado: `13` nomes — André Luiz, Carlos Eduardo, Gustavo Alexandre, Gustavo Augusto, João Pedro, João Vitor, Luis Otávio, Lucas Matheus, Luis Gustavo, Marco Antônio, Pedro Junio e os dois Victor Hugo.

Nível 4 — `find()`, `findIndex()`, `some()` e `every()`

13. Crachá rasgado

Um cliente achou um crachá rasgado perto do estacionamento. Só dá para ler o começo do nome: `"Nic"`. Use `find()` com `startsWith()` em `equipe` e exiba `"O crachá é de Nickson"`.

Depois, tente com `"Zé"`. Se o resultado for `undefined`, exiba `"Crachá não pertence a ninguém da equipe"`.

Resultado esperado: `"Nickson"` e, para `"Zé"`, a mensagem de não encontrado.

14. Otávio ou Octavio?

A loja tem dois funcionários com nomes parecidos e o pagamento vive indo para a pessoa errada. Use `find()` para buscar exatamente `"Octavio"` e depois `"Otavio"` (sem acento e sem C). Em seguida, use `findIndex()` para ver a posição de `"Otávio"` e de `"Octavio"` em `equipe`.

Resultado esperado: `"Octavio"` → `"Octavio"`; `"Otavio"` → `undefined`. Posições: Octavio → `37`, Otávio → `38`.

Para discutir: por que a busca por texto precisa ser exata? O que o sistema poderia usar no lugar do nome para evitar a confusão?

Bônus: use `filter()` com `includes("Otávio")` em `equipe`. Por que aparece mais de um nome? Resultado: `["Luis Otávio", "Otávio"]`.

15. Checagem de luvas no açougue

```
const equipeAcougue = ["Otávio", "Gustavo Augusto", "Pedro Junio"];
const usandoLuva = [true, true, false];
```

Cada posição de `usandoLuva` corresponde à mesma posição de `equipeAcougue`. Use `every()` para saber se todos estão de luva e `some()` para saber se alguém esqueceu. Por fim, use `findIndex()` para achar a posição do `false` e descubra quem foi.

Resultado esperado: `every` → `false`; `some` → `true`; posição `2` → `"Pedro Junio"`.

16. Fila do almoço

```
const filaAlmoco = ["Davi", "Erick", "Vinícius", "Heloa", "Nickson"];
```

Use `findIndex()` para achar a posição do Vinícius e exiba `"Vinícius é o 3º da fila"` (lembre que o índice começa em 0). Depois procure o Breno.

Resultado esperado: Vinícius → índice `2`; Breno → `-1`, porque ele não está na fila.

Nível 5 — `reduce()`

17. Vaquinha do bolo

É aniversário do Eduardo e a equipe fez uma vaquinha para o bolo:

```
const vaquinha = [10, 15, 20, 5, 10];
```

Use `reduce()` para somar o valor arrecadado. O bolo custa R$ 80: calcule quanto ainda falta.

Resultado esperado: arrecadado `60`; faltam `20`.

18. Melhor dia da Heloa

A Heloa anotou quantos cartões fidelidade cadastrou de segunda a sexta:

```
const cadastrosHeloa = [14, 9, 21, 17, 11];
```

Use `reduce()` para encontrar o maior número de cadastros. Depois, use `indexOf()` para descobrir em que dia foi.

Resultado esperado: `21`, na posição `2` — quarta-feira.

19. Plantões do fim de semana

```
const plantoes = ["Emerson", "Jorge", "Emerson", "Eris", "Jorge", "Emerson",
"Rafael"];
```

Use `reduce()` para contar quantos plantões o Emerson fez neste mês. Depois, faça o mesmo para o Jorge.

Resultado esperado: Emerson → `3`; Jorge → `2`.

Nível 6 — Combinando métodos

20. Pagamento das horas extras

Use o array `horasExtras` do exercício 11. Cada hora extra vale R$ 25.

1. Com `filter()`, remova os registros com `0` hora.
2. Com `map()`, transforme cada registro no valor a pagar.
3. Com `reduce()`, some o total que a loja vai pagar.

Resultado esperado: 14 registros; o array de valores começa com `[150, 50, 200, 125, …]`; total de `2000` reais.

21. Mini projeto — fechamento do caixa do Vinícius

Estas são as vendas do caixa do Vinícius no sábado. Cada `0` é uma venda cancelada.

```
const vendas = [120, 45, 0, 300, 80, 0, 15, 210];
```

Monte o relatório de fechamento:

1. Com `filter()`, crie `vendasValidas` sem os cancelamentos.
2. Descubra quantas vendas foram canceladas.
3. Com `map()`, transforme as vendas válidas em textos como `"R$ 120"`.
4. Com `find()`, encontre a primeira venda acima de R$ 100.
5. Com `some()`, verifique se houve alguma venda acima de R$ 250.
6. Com `reduce()`, calcule o total vendido e a maior venda.
7. Exiba uma mensagem final com o número de vendas e o total.

Resultado esperado:

- `vendasValidas` → `[120, 45, 300, 80, 15, 210]`
- Canceladas → `2`
- Textos → `["R$ 120", "R$ 45", "R$ 300", "R$ 80", "R$ 15", "R$ 210"]`
- Primeira acima de 100 → `120`
- Alguma acima de 250 → `true`
- Total → `770`; maior venda → `300`
- Mensagem → `"Vinícius fez 6 vendas, total de R$ 770"`